Corrigiendo inversiones, una a la vez

Adaptación a C#

Una lista sin ordenar es solo una lista. Una lista ordenada es otra bestia: puedes hacerle búsqueda binaria en $O(\log n)$ (lo viste en el capítulo anterior); puedes encontrar duplicados recorriéndola una vez en $O(n)$; puedes calcular medianas, percentiles y cualquier consulta de rango en tiempo que depende solo de la respuesta, no de los datos. Ordenar es el acto de concederle a una lista esos superpoderes, y el coste se paga una sola vez.

Tres ordenamientos clásicos — selección, inserción, burbuja — son todos cuadráticos por casi la misma razón profunda. Una *inversión* es un par de posiciones (i, j) con $i < j$ pero $\texttt{items}[i] > \texttt{items}[j]$ — la evidencia local de que una lista no está ordenada. Todo el capítulo trata de cómo eliminarlas.

La misma imagen lleva al siguiente capítulo, con un giro. Los ordenamientos de aquí arreglan las inversiones una a una; los del siguiente arreglan muchas a la vez. El factor entre $O(n^2)$ y $O(n \log n)$ es exactamente la brecha entre esas dos estrategias.

1. Inversiones: la distancia al orden

Una inversión es un par de índices que está fuera de orden. Formalmente: dada una secuencia `items` y un ordenamiento, una inversión es un par (i, j) con $i < j$ y $\texttt{items}[i] > \texttt{items}[j]$.

Una secuencia ordenada tiene cero inversiones — todo par está en orden, por definición. Una secuencia ordenada al revés tiene el máximo: todo par está fuera de orden, lo que da $\binom{n}{2} = n(n-1)/2$ inversiones. Una secuencia aleatoria se sitúa más o menos en el medio: una permutación uniformemente aleatoria tiene una esperanza de $n(n-1)/4$ inversiones, la mitad del máximo, porque cada uno de los $\binom{n}{2}$ pares tiene la misma probabilidad de estar ordenado o no.

Así que el conteo de inversiones es una *distancia al orden*. Va de cero (ya está) a $n(n-1)/2$ (lo peor posible). Ordenar un arreglo significa llevar su conteo de inversiones a cero. La elección de algoritmo es la elección de cómo las eliminas.

La forma de fuerza bruta de contar inversiones es mirar todos los pares:

```
public static int CountInversions<T>(
    IReadOnlyList<T> items, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    int n = items.Count;
    int count = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (order(items[i], items[j]) > 0)
                count++;
        }
    }
    return count;
}
```

Listing 1: Conteo de inversiones por fuerza bruta

Es $O(n^2)$, lo cual está bien — contar inversiones va a ser una ayuda didáctica en este capítulo, no una herramienta de producción. (Hay una forma $O(n \log n)$ de contarlas como efecto secundario del merge sort; la verás en el siguiente capítulo.)

```
Console.WriteLine(Basic.CountInversions(new[] { 1, 2, 3, 4, 5 })); // 0 ya ordenado
Console.WriteLine(Basic.CountInversions(new[] { 5, 4, 3, 2, 1 })); // 10 inverso: n(n-1)/2
Console.WriteLine(Basic.CountInversions(new[] { 3, 1, 4, 1, 5, 9, 2, 6 })); // 8
```

```
0
10
8
```

2. Ordenamiento por selección: encontrar el extremo, repetir

La forma más directa de ordenar: párate en la posición 0 y pregunta "¿qué debería ir aquí?" La respuesta es el mínimo del arreglo entero. Encuéntralo, cámbialo a la posición 0, luego repite para la posición 1 (el mínimo de lo que queda), la posición 2, y así sucesivamente.

```
public static void SelectionSort<T>(
    IList<T> items, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    int n = items.Count;
    for (int i = 0; i < n; i++)
    {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
        {
            if (order(items[j], items[minIdx]) < 0)
                minIdx = j;
        }
        (items[i], items[minIdx]) = (items[minIdx], items[i]);
    }
}
```

Listing 2: Ordenamiento por selección

El bucle interno es exactamente el patrón de encontrar el mínimo del capítulo 1, acotado a la cola sin ordenar. El bucle externo se ejecuta n veces; el interno se ejecuta $n - i - 1$ veces en la iteración i. Comparaciones totales: $1 + 2 + \cdots + (n-1) = n(n-1)/2$. Eso es $O(n^2)$, independientemente de la entrada — el ordenamiento por selección hace el mismo número de comparaciones sobre un arreglo ordenado que sobre uno aleatorio.

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
Basic.SelectionSort(items);
Console.WriteLine(string.Join(", ", items)); // 1, 1, 2, 3, 4, 5, 6, 9
```

```
1, 1, 2, 3, 4, 5, 6, 9
```

El ordenamiento por selección intercambia `items[i]` con `items[minIdx]`, lo cual puede ser un *intercambio largo* — `minIdx` podría estar al final del arreglo. Eso importa para el análisis de inversiones. Un intercambio largo puede cambiar el conteo de inversiones por mucho más de uno, en cualquier dirección. Así que el ordenamiento por selección no cuenta una historia limpia

de inversiones. Su $O(n^2)$ viene de un argumento distinto: n pasadas, cada una haciendo $O(n)$ trabajo para encontrar un extremo. La historia de las inversiones pertenece a los dos algoritmos siguientes.

3. Ordenamiento por inserción: cada elemento encuentra su lugar

El algoritmo clásico de "cómo ordenas una mano de cartas". Tomas la segunda carta; la colocas antes o después de la primera. Tomas la tercera; la deslizas a la izquierda hasta que esté en el lugar correcto. Tomas la cuarta; la misma idea. En la iteración i, el prefijo `items[0..i]` ya está ordenado, y tu único trabajo es insertar `items[i]` en su lugar correcto entre ellos.

La inserción ocurre mediante intercambios adyacentes repetidos. Comparas `items[i]` con su vecino izquierdo; si es menor, los intercambias, luego comparas el resultado con su nuevo vecino izquierdo; sigues hasta que el elemento alcanza el índice 0 o se encuentra con algo que no es menor que él.

```
public static void InsertionSort<T>(
    IList<T> items, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    for (int i = 1; i < items.Count; i++)
    {
        int j = i;
        while (j > 0 && order(items[j], items[j - 1]) < 0)
        {
            (items[j], items[j - 1]) = (items[j - 1], items[j]);
            j--;
        }
    }
}
```

Listing 3: Ordenamiento por inserción

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
Basic.InsertionSort(items);
Console.WriteLine(string.Join(", ", items)); // 1, 1, 2, 3, 4, 5, 6, 9

// Sobre una entrada ya ordenada, no hace ningun intercambio:
var sortedItems = new[] { 1, 2, 3, 4, 5 };
Basic.InsertionSort(sortedItems);
Console.WriteLine(string.Join(", ", sortedItems)); // 1, 2, 3, 4, 5
```

```
1, 1, 2, 3, 4, 5, 6, 9
1, 2, 3, 4, 5
```

Cada intercambio en el ordenamiento por inserción es un intercambio adyacente. Y un intercambio adyacente arregla exactamente una inversión — el par que acabas de desintercambiar. (¿Dos elementos no adyacentes que estaban invertidos? Siguen invertidos después, ya que no tocaste su par.)

Así que el número total de intercambios que realiza el ordenamiento por inserción es *exactamente* el número de inversiones en el arreglo original.

```
public static (T[] sorted, int swaps) InsertionSortCounting<T>(
    IReadOnlyList<T> input, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    var items = (T[])input.ToArray().Clone();
```

```
    int swaps = 0;
    for (int i = 1; i < items.Length; i++)
    {
        int j = i;
        while (j > 0 && order(items[j], items[j - 1]) < 0)
        {
            (items[j], items[j - 1]) = (items[j - 1], items[j]);
            j--;
            swaps++;
        }
    }
    return (items, swaps);
}
```

Listing 4: Ordenamiento por inserción contando intercambios

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
int inversionsBefore = Basic.CountInversions(items);
var (sortedItems, swaps) = Basic.InsertionSortCounting(items);
Console.WriteLine($"inversiones en la entrada: {inversionsBefore}"); // 8
Console.WriteLine($"intercambios realizados:    {swaps}");            // 8
Console.WriteLine($"coinciden:                  {inversionsBefore == swaps}"); // True
```

```
inversiones en la entrada: 8
intercambios realizados:    8
coinciden:                  True
```

Eso no es una coincidencia y no es específico de esta entrada. El conteo de intercambios del ordenamiento por inserción es exactamente el conteo de inversiones de su entrada; el tiempo de ejecución lo sigue dentro de un factor constante. Sobre una entrada ya ordenada (cero inversiones), el `while` interno nunca entra y el ordenamiento por inserción corre en $O(n)$ — es el mejor de los tres en ese caso. Sobre una entrada ordenada al revés ($n(n-1)/2$ inversiones), hace el trabajo máximo. El coste del ordenamiento por inserción es la distancia de la entrada al orden, medida en inversiones, hecha literal.

4. Ordenamiento de burbuja: la historia de inversiones más limpia

La misma disciplina que el ordenamiento por inserción, distinto orden de recorrido. Recorre el arreglo de izquierda a derecha; cada vez que dos elementos adyacentes estén fuera de orden, intercámbialos. Tras una pasada, el elemento más grande ha burbujeado hasta el final. Luego recorre de nuevo el prefijo sin ordenar restante. Tras k pasadas, los k elementos más grandes están al final, en sus posiciones finales.

```
public static void BubbleSort<T>(
    IList<T> items, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    int n = items.Count;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
```

```
        {
            if (order(items[j], items[j + 1]) > 0)
                (items[j], items[j + 1]) = (items[j + 1], items[j]);
        }
    }
}
```

Listing 5: Ordenamiento de burbuja

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
Basic.BubbleSort(items);
Console.WriteLine(string.Join(", ", items)); // 1, 1, 2, 3, 4, 5, 6, 9
```

```
1, 1, 2, 3, 4, 5, 6, 9
```

El ordenamiento de burbuja, como el de inserción, solo intercambia elementos adyacentes. Así que el mismo lema aplica: intercambios totales = inversiones iniciales.

```
public static (T[] sorted, int swaps) BubbleSortCounting<T>(
    IReadOnlyList<T> input, Comparison<T> order = null)
{
    order ??= Comparer<T>.Default.Compare;
    var items = (T[])input.ToArray().Clone();
    int swaps = 0;
    int n = items.Length;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (order(items[j], items[j + 1]) > 0)
            {
                (items[j], items[j + 1]) = (items[j + 1], items[j]);
                swaps++;
            }
        }
    }
    return (items, swaps);
}
```

Listing 6: Ordenamiento de burbuja contando intercambios

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6 };
int inversionsBefore = Basic.CountInversions(items);
var (_, swaps) = Basic.BubbleSortCounting(items);
Console.WriteLine($"inversiones en la entrada: {inversionsBefore}"); //
    8
Console.WriteLine($"intercambios de burbuja:    {swaps}");            //
     8
```

```
inversiones en la entrada: 8
intercambios de burbuja:    8
```

El mismo número. Ese es el invariante hermoso: el ordenamiento por inserción y el de burbuja hacen exactamente el mismo número de intercambios sobre cualquier entrada, porque ambos están restringidos a intercambios adyacentes, y cada intercambio adyacente arregla exactamente una inversión. La diferencia entre ellos está solo en *dónde* ocurre cada intercambio — el de

inserción hace su trabajo localmente mientras escanea; el de burbuja lo hace globalmente en cada pasada. La cantidad total de trabajo, medida en intercambios, es idéntica.

Eso es un hecho más profundo de lo que parece. La elección entre ordenamiento por inserción y de burbuja no es cuestión de eficiencia. Es cuestión de orden de recorrido. Están resolviendo el mismo problema, pagando el mismo precio, en la misma moneda.

5. Las tres preguntas, aplicadas

¿Es correcto?

Los tres ordenamientos tienen invariantes de bucle limpios.

- **Ordenamiento por selección.** Al inicio de la iteración i, el prefijo `items[0..i]` contiene los i elementos más pequeños en orden. La iteración encuentra el mínimo de `items[i..]` y lo coloca en la posición i, extendiendo el prefijo ordenado en uno. Tras n iteraciones, el arreglo entero es el prefijo ordenado.
- **Ordenamiento por inserción.** Al inicio de la iteración i, el prefijo `items[0..i]` está ordenado (pero no necesariamente con sus valores finales — puede desplazarse más a medida que llegan elementos posteriores). La iteración inserta `items[i]` en su lugar correcto dentro de ese prefijo, así que `items[0..i+1]` está ordenado al final.
- **Ordenamiento de burbuja.** Tras la pasada k, el sufijo `items[n-k..n]` contiene los k elementos más grandes en orden, en sus posiciones finales. La siguiente pasada recorre solo el prefijo sin ordenar y burbujea su elemento más grande hasta la frontera.

En cada caso el invariante se cumple al inicio, se preserva en cada iteración, y al terminar implica que el arreglo entero está ordenado.

¿Cuán eficiente es?

Los tres son tiempo $O(n^2)$ y espacio extra $O(1)$ — los algoritmos ordenan *in situ*, modificando la entrada directamente. El detalle es interesante, sin embargo, y divide a los algoritmos en dos bandos.

- **Ordenamiento por selección** hace exactamente $n(n-1)/2$ comparaciones sobre toda entrada. La estructura de la entrada — ordenada, inversa, aleatoria — no afecta el conteo de comparaciones. Siempre hace n pasadas, cada una haciendo $O(n)$ trabajo para encontrar el mínimo. Mejor caso = peor caso = caso promedio.
- **Ordenamiento por inserción y de burbuja** hacen un número de intercambios igual al conteo de inversiones de la entrada. Sobre entrada ordenada eso es cero intercambios, dando tiempo $O(n)$ (todavía necesitas el bucle externo para verificar, y las comparaciones ocurren). Sobre entrada ordenada al revés son $n(n-1)/2$ intercambios, dando $O(n^2)$. Sobre entrada aleatoria son unos $n^2/4$ intercambios, todavía $O(n^2)$.

Dicho de otra forma: el ordenamiento por selección es trabajo constante, independientemente de la suerte. El de inserción y el de burbuja trabajan tanto como la entrada merece — lo cual, para una entrada aleatoria, es la misma cantidad en promedio, pero ganan mucho en entradas que están casi ordenadas.

¿Es óptimo?

Este es el golpe del capítulo, y se divide en dos afirmaciones claramente distintas.

Afirmación 1. Cualquier algoritmo de ordenamiento basado en comparaciones que use solo intercambios adyacentes debe hacer al menos tantos intercambios como inversiones haya en la entrada. En el peor caso (entrada ordenada al revés), eso es $n(n-1)/2$. Así que el ordenamiento por inserción y el de burbuja son ajustados en $\Omega(n^2)$ intercambios en el peor caso — no hay reordenamiento astuto dentro de la disciplina de intercambios adyacentes que te baje del conteo de inversiones.

Lema 1. *Un intercambio adyacente cambia el conteo de inversiones en exactamente* ± 1*. Para llegar de cualquier conteo inicial de inversiones* k *a cero, necesitas al menos* k *intercambios adyacentes.*

El peor caso de k es $n(n-1)/2$. Fin.

Afirmación 2. El ordenamiento por selección no está restringido a intercambios adyacentes — hace intercambios largos — así que el argumento de las inversiones no le aplica. Su $O(n^2)$ viene de un hecho estructural distinto: para identificar el mínimo de un subarreglo de tamaño m, necesitas $m-1$ comparaciones (ningún algoritmo basado en comparaciones puede hacerlo mejor; el mismo argumento del adversario del capítulo 1). El ordenamiento por selección hace esto n veces para subarreglos de tamaños $n, n-1, n-2, \ldots, 1$. Total: $\Theta(n^2)$ comparaciones, inevitablemente.

Teorema 1. *El ordenamiento por selección realiza* $\Theta(n^2)$ *comparaciones en todo caso, y el ordenamiento por inserción y el de burbuja realizan* $\Omega(n^2)$ *intercambios en el peor caso. Ambos están en* $\Omega(n^2)$*.*

Ambas afirmaciones aterrizan en el mismo lugar — $\Omega(n^2)$ — pero por razones distintas. El de inserción y el de burbuja están atascados porque solo arreglan una inversión por operación. El de selección está atascado porque no puede atajar el trabajo de encontrar el extremo. El siguiente capítulo rompe ambas restricciones a la vez.

6. Distancia al orden

Los ordenamientos por selección, inserción y burbuja son todos correctos — tres invariantes de bucle ligeramente distintos para lo que moralmente es la misma forma. Los tres corren en tiempo $O(n^2)$ y espacio $O(1)$. Y los tres son óptimos dentro de sus respectivas restricciones: el de inserción y el de burbuja están atascados en $\Omega(n^2)$ porque solo intercambian elementos adyacentes; el de selección está atascado porque no puede vencer la cota inferior de encontrar el extremo — pero esas restricciones son exactamente lo que un algoritmo puede abandonar para hacerlo mejor.

El conteo de inversiones mide la distancia al orden. Un arreglo sin ordenar está a un número específico de intercambios de pares de estar ordenado, y ese número acota cuánto trabajo tiene que hacer cualquier algoritmo local. La razón por la que los ordenamientos cuadráticos son cuadráticos es que arreglan una inversión a la vez, y puede haber un número cuadrático de inversiones.

La salida es arreglar muchas inversiones por operación. El *merge sort* lo hace combinando dos mitades ordenadas y resolviendo todas las inversiones cruzadas en una sola pasada lineal. El *quicksort* lo hace particionando alrededor de un pivote, lo que coloca un elemento en su posición final y resuelve todas las inversiones que involucran a ese pivote en un solo intercambio. Ambos algoritmos rompen el techo de una inversión por intercambio, y ambos lo pagan con un factor $\log n$ en lugar de un factor n.

7. Código completo

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace Codex.Sort
{
    public static class Basic
    {
        // ---------- Conteo de inversiones (fuerza bruta) ----------
        public static int CountInversions<T>(
            IReadOnlyList<T> items, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            int n = items.Count;
            int count = 0;
            for (int i = 0; i < n; i++)
                for (int j = i + 1; j < n; j++)
                    if (order(items[i], items[j]) > 0)
                        count++;
            return count;
        }

        // ---------- Ordenamiento por selecci n ----------
        public static void SelectionSort<T>(
            IList<T> items, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            int n = items.Count;
            for (int i = 0; i < n; i++)
            {
                int minIdx = i;
                for (int j = i + 1; j < n; j++)
                    if (order(items[j], items[minIdx]) < 0)
                        minIdx = j;
                (items[i], items[minIdx]) = (items[minIdx], items[i]);
            }
        }

        // ---------- Ordenamiento por inserci n ----------
        public static void InsertionSort<T>(
            IList<T> items, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            for (int i = 1; i < items.Count; i++)
            {
                int j = i;
                while (j > 0 && order(items[j], items[j - 1]) < 0)
                {
                    (items[j], items[j - 1]) = (items[j - 1], items[j]);
                    j--;
                }
            }
        }

        public static (T[] sorted, int swaps) InsertionSortCounting<T>(
            IReadOnlyList<T> input, Comparison<T> order = null)
```

```
        {
            order ??= Comparer<T>.Default.Compare;
            var items = input.ToArray();
            int swaps = 0;
            for (int i = 1; i < items.Length; i++)
            {
                int j = i;
                while (j > 0 && order(items[j], items[j - 1]) < 0)
                {
                    (items[j], items[j - 1]) = (items[j - 1], items[j]);
                    j--;
                    swaps++;
                }
            }
            return (items, swaps);
        }

        // ---------- Ordenamiento de burbuja ----------
        public static void BubbleSort<T>(
            IList<T> items, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            int n = items.Count;
            for (int i = 0; i < n - 1; i++)
                for (int j = 0; j < n - 1 - i; j++)
                    if (order(items[j], items[j + 1]) > 0)
                        (items[j], items[j + 1]) = (items[j + 1], items[
                            j]);
        }

        public static (T[] sorted, int swaps) BubbleSortCounting<T>(
            IReadOnlyList<T> input, Comparison<T> order = null)
        {
            order ??= Comparer<T>.Default.Compare;
            var items = input.ToArray();
            int swaps = 0;
            int n = items.Length;
            for (int i = 0; i < n - 1; i++)
                for (int j = 0; j < n - 1 - i; j++)
                    if (order(items[j], items[j + 1]) > 0)
                    {
                        (items[j], items[j + 1]) = (items[j + 1], items[
                            j]);
                        swaps++;
                    }
            return (items, swaps);
        }
    }
}
```

Listing 7: `Basic.cs` — todos los algoritmos

8. Notas y lecturas adicionales

El conteo de inversiones como propiedad estructural de las permutaciones se trata a fondo en *The Art of Computer Programming* Vol. 3 §5.1.1 de Knuth, donde aparece bajo el nombre de "inversiones de una permutación". Los ordenamientos por selección, inserción y burbuja están en

CLRS (4.ª ed.) §2.1 y §2.2 y en *Algorithms* (4.ª ed.) de Sedgewick y Wayne §2.1. La equivalencia entre conteos de intercambios adyacentes y conteos de inversiones es folklore pero vale la pena atribuirla — es el hecho estructural que hace al ordenamiento por inserción la elección correcta para entradas casi ordenadas, razón por la cual se usa como caso base dentro de la mayoría de ordenamientos híbridos de producción (Timsort, Introsort). Contar inversiones en $O(n \log n)$ vía merge sort es un ejercicio estándar que encontrarás en el siguiente capítulo.